Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ОТ

Звіт

з лабораторної роботи № 4 з дисципліни
«Бази даних»

«Аналітичні SQL-запити (OLAP)»

Виконали:
ІМ-51 Господінова Марія Михайлівна
ІМ-51 Резник Валерія Вадимівна

Перевірив: Хмельницький Арсеній Андрійович

Київ 2026

Лабораторна робота №4. Аналітичні SQL-запити (OLAP)

Мета:
Отримання практичних навичок складання складних аналітичних SQL-запитів у парадигмі OLAP (Online Analytical Processing) у СУБД PostgreSQL для сервісу Wishlist. Опанування агрегатних функцій (COUNT, SUM, AVG, MIN, MAX), групування даних за допомогою GROUP BY та фільтрації агрегованих значень через HAVING, використання різних типів з'єднань таблиць (INNER JOIN, LEFT JOIN, FULL JOIN) та вкладених підзапитів для формування зведеної статистики щодо користувачів, списків бажань, товарів і бронювань, а також проведення аналітики популярності товарів та активності користувачів.

Завдання:

1. Розробити та протестувати аналітичні запити з використанням базових та багатовимірних агрегатних функцій.
2. Реалізувати групування даних за категоріями з фільтрацією груп за допомогою речення HAVING.
3. Скласти запити з використанням різних типів реляційних з'єднань (INNER JOIN, LEFT JOIN, FULL JOIN) для аналізу пов'язаних сутностей.
4. Розробити аналітичні запити з використанням підзапитів у блоках SELECT, WHERE та HAVING.
5. Зафіксувати результати виконання запитів у середовищі pgAdmin 4 за допомогою скріншотів.

Опис реалізованих OLAP-сценаріїв:

1. Базова агрегація.
Реалізовано отримання загальної статистики за всіма товарами. Запит визначає кількість товарів, загальну суму їхніх цін, середню, мінімальну та максимальну ціну.

2. Групування.
Реалізовано отримання статистики за кожним списком бажань, який містить хоча б один товар. Для кожного списку визначається назва, кількість товарів, загальна та середня ціна. Результати сортуються за сумою цін у порядку спадання.

3. Фільтр груп.
Реалізовано пошук списків бажань, загальна вартість товарів у яких перевищує 150. Для цього товари групуються за списками, після чого залишаються лише групи, які відповідають заданій умові.

4. Агрегація по кількох таблицях.
Реалізовано отримання статистики за кожним користувачем. Запит визначає ім'я користувача, кількість його списків бажань, загальну кількість товарів та їхню сумарну вартість. До результату потрапляють також користувачі без списків або товарів. Для них сумарна вартість відображається як 0.

5. INNER JOIN.
Реалізовано отримання інформації про товари та їхніх власників. Для кожного товару виводиться його назва, ціна, назва списку бажань та ім'я користувача, якому належить цей список. Товари, які не пов'язані зі списками або користувачами, до результату не потрапляють.

6. LEFT JOIN.
Реалізовано отримання інформації про всіх користувачів, кількість їхніх списків бажань та середню ціну товарів. Завдяки LEFT JOIN у результаті залишаються навіть користувачі, які не мають списків. Для підрахунку кількості списків використовується COUNT(DISTINCT ...), щоб уникнути повторного підрахунку одного списку через з'єднання з товарами.

7. RIGHT JOIN.
Реалізовано отримання інформації про всіх користувачів та назви товарів,

які вони зарезервували. Користувачі, які не мають резервацій, також потрапляють до результату, але назва товару для них буде NULL.

8. Підзапит у WHERE.
Реалізовано пошук товарів, ціна яких перевищує середню ціну всіх товарів у базі даних. Середня ціна обчислюється у вкладеному запиті, а основний запит відбирає товари, дорожчі за це значення.

9. Підзапит у SELECT.
Реалізовано отримання інформації про кожен список бажань із зазначенням його назви, імені власника та кількості товарів. Кількість товарів визначається за допомогою скалярного підзапиту. Завдяки цьому до результату потрапляють також порожні списки, для яких кількість товарів дорівнює 0.

10. Підзапит у HAVING.
Реалізовано пошук списків бажань, середня ціна товарів у яких перевищує середню ціну всіх товарів у базі даних. Для кожного списку обчислюється середня ціна, яка порівнюється із загальною середньою ціною за допомогою підзапиту.

11. Агрегація.
Реалізовано аналіз цін товарів у кожному списку бажань. Для кожного списку визначається мінімальна та максимальна ціна товарів, а також різниця між ними. Це дозволяє визначити, наскільки найдорожчий товар дорожчий за найдешевший.

12. CROSS JOIN.
Реалізовано формування всіх можливих пар «користувач — публічний список», де список належить іншому користувачу. Спочатку створюються всі можливі комбінації користувачів і списків, після чого залишаються лише пари, які відповідають заданим умовам. Результати сортуються за іменами користувачів.

13. Підзапит у WHERE (IN та EXISTS).
Реалізовано пошук користувачів, які зарезервували хоча б один товар. Для цього використано два варіанти підзапитів: IN та EXISTS. Обидва запити дозволяють отримати імена користувачів, для яких існує хоча б один запис у таблиці резервацій.

Повний SQL код (текстом і скріншотом):

```
-- 1.
SELECT
COUNT(*) AS items_count,
SUM(price) AS total_price,
ROUND(AVG(price), 2) AS avg_price,
MIN(price) AS min_price,
MAX(price) AS max_price
FROM wishlist_items
-- 2.
SELECT
w.name AS wishlist_name,
COUNT(wi.wishlist_id) AS items_count,
SUM(wi.price) AS sum_price,
ROUND(AVG(wi.price), 2) AS avg_price
FROM wishlists AS w
JOIN wishlist_items AS wi
ON w.wishlist_id = wi.wishlist_id
GROUP BY w.wishlist_id
ORDER BY sum_price DESC
-- 3.
SELECT
w.wishlist_id,
w.name AS wishlist_name,
```

```
SUM(wi.price) AS sum_price
FROM wishlists AS w
JOIN wishlist_items AS wi
ON w.wishlist_id = wi.wishlist_id
GROUP BY w.wishlist_id
HAVING SUM(wi.price) > 150
-- 4.
SELECT
u.name AS user_name,
COUNT(w.wishlist_id) AS wishlists_quantity,
COUNT(wi.item_id) AS wishlist_items_quantity,
COALESCE(SUM(wi.price), 0) AS total_price
FROM users AS u
LEFT JOIN wishlists AS w
ON u.user_id = w.user_id
LEFT JOIN wishlist_items AS wi
ON w.wishlist_id = wi.wishlist_id
GROUP BY u.user_id
ORDER BY total_price DESC
-- 5.
SELECT
wi.name AS wishlist_item_name,
wi.price AS wishlist_item_price,
w.name AS wishlist_name,
```

```
u.name AS user_name
FROM wishlist_items AS wi
JOIN wishlists AS w
ON w.wishlist_id = wi.wishlist_id
JOIN users AS u
ON w.user_id = u.user_id
-- 6.
SELECT
u.name AS user_name,
COUNT(DISTINCT w.wishlist_id) AS wishlists_quantity,
ROUND(AVG(wi.price), 2) AS avarage_items_price
FROM users AS u
LEFT JOIN wishlists AS w
ON u.user_id = w.user_id
LEFT JOIN wishlist_items AS wi
ON w.wishlist_id = wi.wishlist_id
GROUP BY u.name
-- 7.
SELECT
u.name AS user_name,
wi.name AS item_name
FROM reservations AS r
RIGHT JOIN wishlist_items AS wi
ON r.item_id = wi.item_id
```

```
RIGHT JOIN users AS u
ON r.user_id = u.user_id
-- 8.
SELECT
name,
price
FROM wishlist_items
WHERE price > (
SELECT AVG(price)
FROM wishlist_items
)
-- 9.
SELECT
w.name AS wishlist_name,
u.name AS user_name,
(
SELECT COUNT(*)
FROM wishlist_items AS wi
WHERE w.wishlist_id = wi.wishlist_id
) AS items_count
FROM wishlists AS w
JOIN users AS u
ON w.user_id = u.user_id
-- 10.
```

```
SELECT
w.name,
ROUND(AVG(wi.price), 2) AS avarage_price
FROM wishlists AS w
JOIN wishlist_items AS wi
ON w.wishlist_id = wi.wishlist_id
GROUP BY w.wishlist_id
HAVING ROUND(AVG(wi.price), 2) > (
SELECT AVG(price)
FROM wishlist_items
)
-- 11.
SELECT
w.name AS wishlist_name,
MIN(wi.price) AS min_price,
MAX(wi.price) AS max_price,
(MAX(wi.price) - MIN(wi.price)) AS price_difference
FROM wishlists AS w
JOIN wishlist_items AS wi
ON w.wishlist_id = wi.wishlist_id
GROUP BY w.wishlist_id
-- 12.
SELECT
u.name AS user_name,
```

```
w.name AS wishlist_name
FROM users AS u
CROSS JOIN wishlists AS w
WHERE u.user_id != w.user_id
AND w.is_public = TRUE
ORDER BY u.name ASC
-- 13.
SELECT name
FROM users
WHERE user_id IN (
SELECT user_id
FROM reservations
)
SELECT name
FROM users AS u
WHERE EXISTS (
SELECT 1
FROM reservations AS r
WHERE r.user_id = u.user_id
);
```

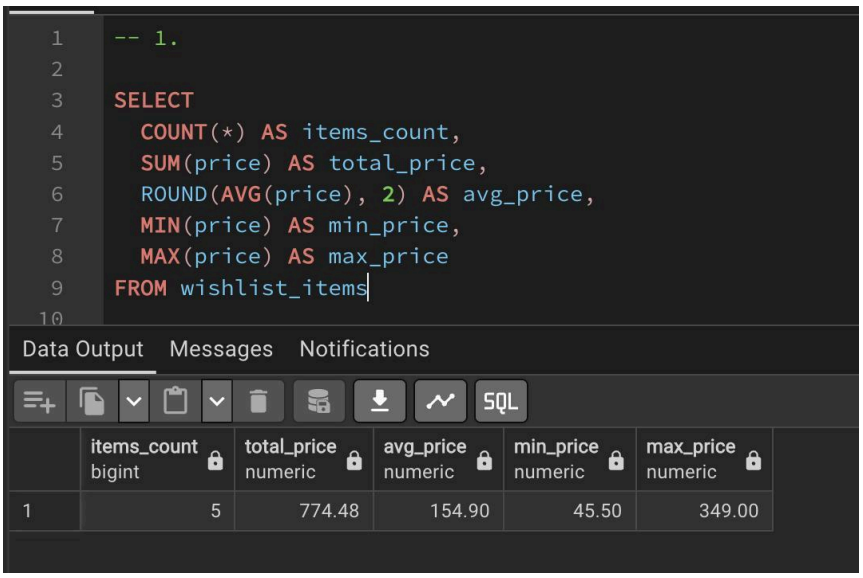


Рис.1 Базова агрегація.

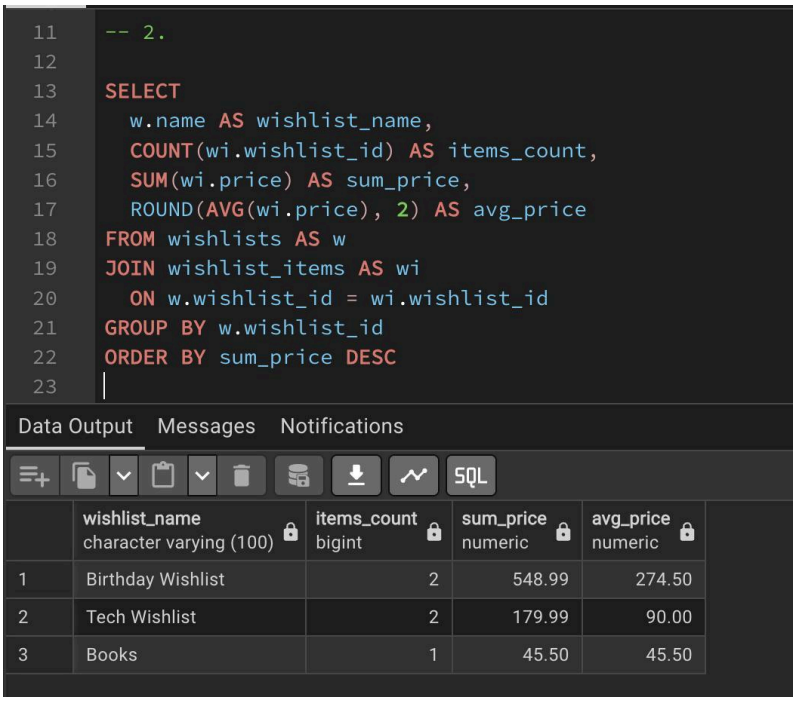


Рис.2 Групування

```sql
-- 3.

SELECT
    w.wishlist_id,
    w.name AS wishlist_name,
    SUM(wi.price) AS sum_price
FROM wishlists AS w
JOIN wishlist_items AS wi
    ON w.wishlist_id = wi.wishlist_id
GROUP BY w.wishlist_id
HAVING SUM(wi.price) > 150

```

Data Output | Messages | Notifications

	wishlist_id [PK] uuid	wishlist_name character varying (100)	sum_price numeric
1	515baa83-ccfc-4020-bbe1-eab66b9ce8...	Birthday Wishlist	548.99
2	7d147718-a074-494e-bc69-527b3ae45...	Tech Wishlist	179.99

Рис.3 Фільтр груп

```sql
-- 4.

SELECT
    u.name AS user_name,
    COUNT(w.wishlist_id) AS wishlists_quantity,
    COUNT(wi.item_id) AS wishlist_items_quantity,
    COALESCE(SUM(wi.price), 0) AS total_price
FROM users AS u
LEFT JOIN wishlists AS w
    ON u.user_id = w.user_id
LEFT JOIN wishlist_items AS wi
    ON w.wishlist_id = wi.wishlist_id
GROUP BY u.user_id
ORDER BY total_price DESC

```

Data Output | Messages | Notifications

	wishlist_id [PK] uuid	wishlist_name character varying (100)	sum_price numeric
1	515baa83-ccfc-4020-bbe1-eab66b9ce8...	Birthday Wishlist	548.99
2	7d147718-a074-494e-bc69-527b3ae45...	Tech Wishlist	179.99

Рис.4 Агрегація по кількох таблицях.

```sql
-- 5.

SELECT
    wi.name AS wishlist_item_name,
    wi.price AS wishlist_item_price,
    w.name AS wishlist_name,
    u.name AS user_name
FROM wishlist_items AS wi
JOIN wishlists AS w
    ON w.wishlist_id = wi.wishlist_id
JOIN users AS u
    ON w.user_id = u.user_id
```

Data Output | Messages | Notifications

Showing rows: 1

	wishlist_item_name character varying (150)	wishlist_item_price numeric (10,2)	wishlist_name character varying (100)	user_name character varying (100)
1	Wireless Headphones	199.99	Birthday Wishlist	Maria
2	Coffee Machine	349.00	Birthday Wishlist	Maria
3	Clean Code	45.50	Books	Maria
4	Mechanical Keyboard	120.00	Tech Wishlist	Anna
5	USB-C Hub	59.99	Tech Wishlist	Anna

Рис.5 INNER JOIN.

```sql
-- 6.

SELECT
    u.name AS user_name,
    COUNT(DISTINCT w.wishlist_id) AS wishlists_quantity,
    ROUND(AVG(wi.price), 2) AS avarage_items_price
FROM users AS u
LEFT JOIN wishlists AS w
    ON u.user_id = w.user_id
LEFT JOIN wishlist_items AS wi
    ON w.wishlist_id = wi.wishlist_id
GROUP BY u.name
```

Data Output | Messages | Notifications

	user_name character varying (100)	wishlists_quantity bigint	avarage_items_price numeric
1	Anna	1	90.00
2	Daria	0	[null]
3	Maria	2	198.16
4	Olena	1	[null]
5	Sofia	0	[null]
6	Valeriia	0	[null]

Рис.6 LEFT JOIN.

```sql
-- 7.

SELECT
    u.name AS user_name,
    wi.name AS item_name
FROM reservations AS r
RIGHT JOIN wishlist_items AS wi
    ON r.item_id = wi.item_id
RIGHT JOIN users AS u
    ON r.user_id = u.user_id

```

Data Output | Messages | Notifications

	user_name character varying (100)	item_name character varying (150)
1	Anna	Wireless Headphones
2	Sofia	Mechanical Keyboard
3	Olena	USB-C Hub
4	Valeriia	[null]
5	Maria	[null]
6	Daria	[null]

Рис.7 RIGHT JOIN.

```sql
-- 8.

SELECT
    name,
    price
FROM wishlist_items
WHERE price > (
    SELECT AVG(price)
    FROM wishlist_items
)
```

Data Output | Messages | Notifications

	name character varying (150)	price numeric (10,2)
1	Wireless Headphones	199.99
2	Coffee Machine	349.00

Рис.8 Підзапит у WHERE.

```sql
-- 9.

SELECT
    w.name AS wishlist_name,
    u.name AS user_name,
    (
        SELECT COUNT(*)
        FROM wishlist_items AS wi
        WHERE w.wishlist_id = wi.wishlist_id
    ) AS items_count
FROM wishlists AS w
JOIN users AS u
    ON w.user_id = u.user_id
```

Data Output | Messages | Notifications

	wishlist_name character varying (100)	user_name character varying (100)	items_count bigint
1	Birthday Wishlist	Maria	2
2	Books	Maria	1
3	Tech Wishlist	Anna	2
4	Private Wishlist	Olena	0

Рис.9 Підзапит у SELECT.

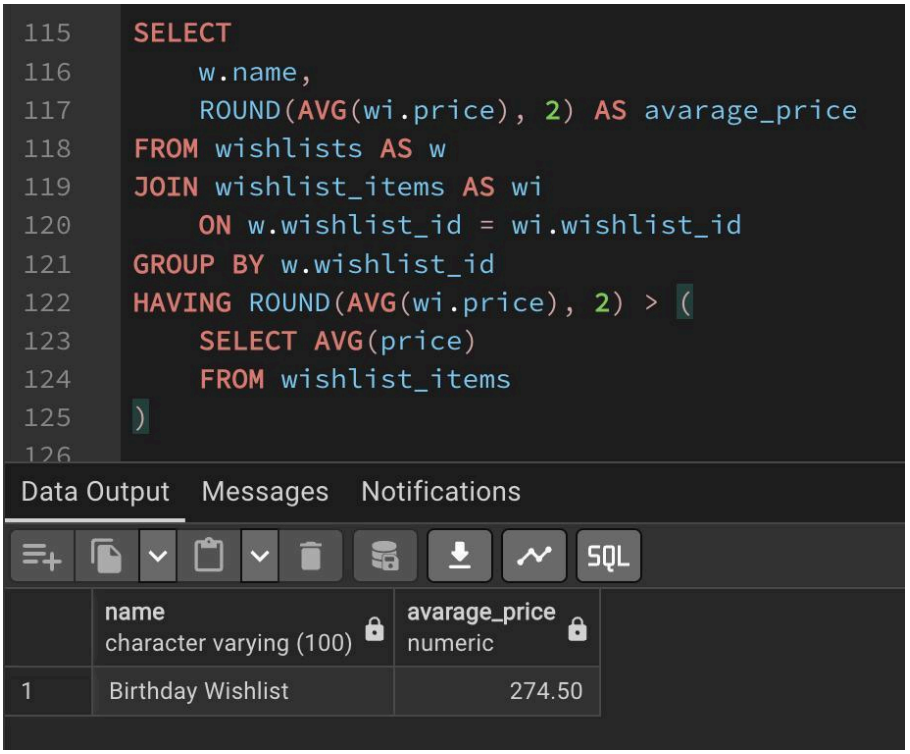


Рис.10 Підзапит у HAVING.

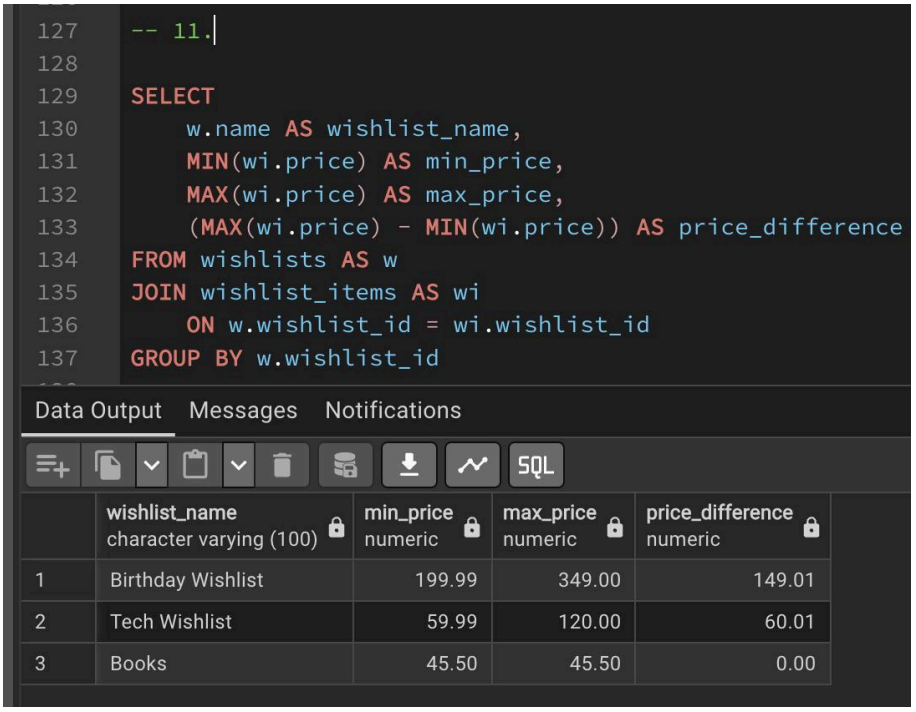


Рис.11 Агрегація

```sql
-- 12.

SELECT
    u.name AS user_name,
    w.name AS wishlist_name
FROM users AS u
CROSS JOIN wishlists AS w
WHERE u.user_id != w.user_id
    AND w.is_public = TRUE
ORDER BY u.name ASC
```

Data Output | Messages | Notifications

	user_name character varying (100)	wishlist_name character varying (100)
1	Anna	Birthday Wishlist
2	Anna	Books
3	Daria	Tech Wishlist
4	Daria	Books
5	Daria	Birthday Wishlist
6	Maria	Tech Wishlist
7	Olena	Birthday Wishlist
8	Olena	Books
9	Olena	Tech Wishlist
10	Sofia	Books
11	Sofia	Birthday Wishlist
12	Sofia	Tech Wishlist

Рис.12 CROSS JOIN.

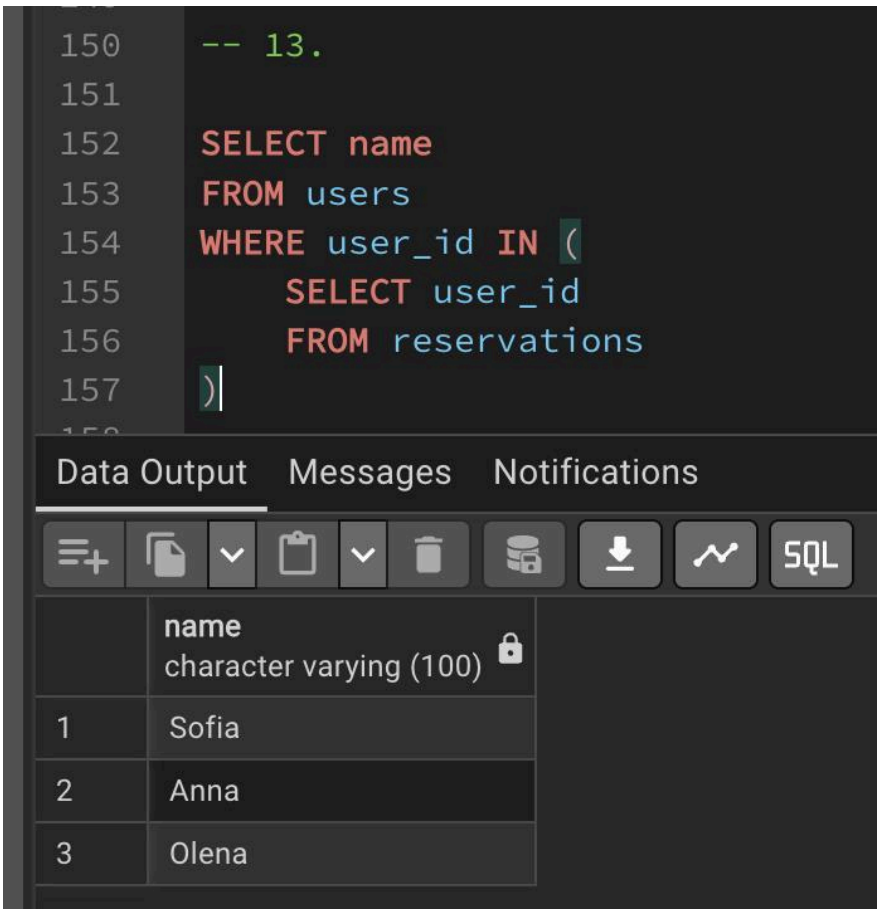


Рис.13 Підзапит у WHERE (IN та EXISTS).

Висновок:

Під час виконання роботи було закріплено навички написання SQL-запитів для отримання та обробки даних із бази даних. Було реалізовано запити з використанням агрегатних функцій (COUNT, SUM, AVG, MIN, MAX), групування (GROUP BY), фільтрації груп (HAVING), сортування (ORDER BY) та різних типів з'єднань таблиць (INNER JOIN, LEFT JOIN, RIGHT JOIN, CROSS JOIN). Також було розглянуто використання підзапитів у конструкціях WHERE, SELECT та HAVING, зокрема з операторами IN і EXISTS.

У результаті роботи отримано практичні навички формування складних запитів, об'єднання даних із кількох таблиць, виконання обчислень та фільтрації результатів відповідно до заданих умов. Це дозволило краще зрозуміти можливості SQL та навчитися використовувати їх для аналізу даних у базі даних.